문제 A

문제요약

- 목표 : 합이 total 이 되는 연속형 정수 num개를 찾아 오름차순으로 출력함.
- 제한 : 1 <=num <=100, 0<=total <= 1000

접근 전략

수학적 모델링 : 첫 번째 수를 (x)로 설정 시, 전체 합은

$$\text{total} = (x \times num) + \frac{num(num-1)}{2}$$

해결 방안

위 식을 역산하여 시작 숫자 x를 즉시 도출하는 O(1)전략 사용

알고리즘 설계 (수도코드)

```
함수 연속된_정수_합_찾기(num, total):
// 1. 오프셋 합계(증가분 총합) 계산
// 수식의 [num(num-1) / 2] 부분을 먼저 계산하여 변수에 저장합니다.
설정 오프셋_합 = (num * (num - 1)) / 2

// 2. 시작 숫자 x 도출 (역산 공식 적용)
// 수식: x = (total - 오프셋_합) / num
// 전체 합계에서 증가분들을 뺀 나머지를 개수로 나누어 첫 항을 찾습니다.
설정 시작_숫자 = (total - 오프셋_합) / num

// 3. 수열 생성 및 저장
// 찾아낸 시작_숫자부터 1씩 증가시키며 num개의 정수를 목록에 담습니다.
설정 결과_리스트 = 빈 목록

반복 i를 0부터 (num - 1)까지 수행:
설정 현재_숫자 = 시작_숫자 + i
결과_리스트에 현재_숫자 추가

// 4. 최종 결과 출력
결과_리스트의 모든 원소를 공백으로 구분하여 한 줄에 출력
종료
```

문제 B

문제요약

- 목표 : N개의 통나무를 원형으로 세워 놓았을 때, 인접한 두 통나무 간의 높이 차의 최댓값(난이도)이 최소가 되도록 배치하여 그 최솟값을 출력함.
- 제한 : 통나무 개수를 나타내는 정수 n(5 <= n <= 100,000), 통나무 높이를 나타내는 정수 Li(1 <= Li <= 100.000)

접근 전략

- 그리디 알고리즘 : 통나무를 단순히 크기순으로 나열하면 마지막과 첫 번째 통나무의 높이 차이가 매우 커짐.
- 배치 원리 : 가장 큰 통나무를 중심으로 양옆에 그다음 큰 나무들을 번갈아 가며 배치하는 '지그재그' 방식을 사용함.
- 수학적 특징 : 최적의 배치를 하면 정렬된 배열에서 인덱스 차이가 2인 원소들 L[i]와 L[i-2]이 서로 인접하게 됨.

해결 방안

입력받은 높이 리스트를 오름차순으로 정렬한 후, 한 칸 건너뛴 원소 간의 차이 L[i] - L[i-2] 중 가장 큰 값을 도출하는 O(N log N) 전략 사용.

알고리즘 설계 (수도코드)

```
시작
// 1. 전체 테스트 케이스(T) 입력
테스트_케이스_수(T)를 입력받음
T번 반복:
// 2. 통나무 개수(N) 입력 및 유효성 검사
N을 입력받음
만약 N이 5 미만이거나 10,000 초과라면:
오류 메시지 출력 후 다음 테스트로 넘어감

// 3. 통나무 높이 데이터 입력 및 필터링
데이터_바구니(trees) 생성 (빈 목록)
입력받은 높이 값들을 하나씩 확인:
숫자로 변환한 높이가 1 이상 100,000 이하라면:
데이터_바구니에 추가
아니라면:
범위 초과 경고 출력

// 입력된 나무 수가 N과 일치하는지 확인
만약 데이터_바구니의 개수가 N과 다르면:
개수 오류 출력 후 다음 테스트로 넘어감

// 4. 핵심 로직: 정렬 및 지그재그 높이 차 계산
데이터_바구니를 오름차순으로 정렬 (작은 순서대로 나열)

최대_난이도 = 0 설정

// 원형 배치를 위해 두 칸 떨어진 나무(인접하게 될 이웃)끼리 비교
반복 (i = 2부터 N-1까지):
현재_차이 = 데이터_바구니[i] - 데이터_바구니[i - 2]
만약 현재_차이가 최대_난이도보다 크다면:
최대_난이도를 현재_차이로 갱신

// 5. 결과 출력
최종 계산된 최대_난이도 출력
종료
```

문제 C

문제요약

- 목표: '좋은 문자열' A를 인접한 두 문자를 바꾸는 연산을 통해 '좋은 문자열' B로 변환할 때 필요한 최소 연산 횟수를 구함.
  좋은 문자열 정의: 1. ab는 좋은 문자열이다. 2. S가 좋은 문자열이면 a[S]b도 좋은 문자열이다.
  3. S와 T가 좋은 문자열이면 [S][T]도 좋은 문자열이다.
- 제한: 문자열 길이 2<= L <= 100,000, 시간 제한 1초. 변환 과정 중의 문자열도 모두 '좋은 문자열'이어야 함.

접근 전략

- 구조 분석: '좋은 문자열'의 정의는 올바른 괄호 문자열(Parenthesis String)과 구조가 동일함. // a를 (, b를)로 치환 가능
- 연산을 할 개수가 적으면 순차탐색을 사용 그렇지 않을경우 이진탐색을 이용
- 제약 조건 해석: 변환 과정 중에도 항상 '좋은 문자열'이어야 한다는 조건은 괄호의 중첩 구조가 깨지지 않아야 함을 의미함.

해결 방안

- 문자열 A와 B에서 각 'a'가 등장하는 위치(인덱스)를 각각 리스트에 담음.
- A의 i 번째 'a'가 B의 i번째 'a' 위치로 이동해야 하므로, 두 위치의 절대값 차이 (posAi - posBi)를 모두 합산하여 O(N)전략 사용

알고리즘 설계 (수도코드)

```
// 1. 전체 테스트 케이스(T) 입력
모든 데이터를 한 번에 읽어와 단어 단위로 리스트 생성
총 풀어야 할 테스트 케이스 수(T)를 확인
T번 반복:

// 2. A, B데이터 입력
입력변환 전 문자열 A와 목표 문자열 B를 순서대로 가져옴

// 3. 각 문자열에 'a'의 위치를 담을 pos_A,B 생성:
pos_A생성: pos_A(문자열)를 처음부터 훑으며
'a'가 나타나는 인덱스 번호를 모두 적음
pos_B 생성: pos_B(문자열)를 처음부터 훑으며
'a'가 나타나는 인덱스 번호를 모두 적음

// 4. 유효성 검사 (개수 체크)
만약 pos_A의 개수와 pos_B의 개수가 다르다면:
개수가 맞지 않아 변환 불가. -1 출력
개수가 맞다면 다음 로직으로 이동.

// 5. 핵심 로직: 구조적 안전성 검사 및 이동 거리 합산
총_연산_횟수 = 0 설정
반복 (i = 0부터 'a'의 개수만큼):
만약 pos_A의 i번째 'a' 위치가 pos_B의 목표 위치보다 앞에 있다면:
'a'를 뒤로 밀면 규칙이 깨지므로 실패 처리 후 반복 중단
아닐 경우: 현재 위치에서 목표 위치를 뺀 값을 총_연산_횟수에 더함

// 6. 결과
계산이 끝났다면 최종 계산된 연산 횟수 출력
아니오(규칙에서 위반될 시) -1 출력:
종료
```

문제 D

문제요약

- 목표: 집합 A의 부분집합과 1보다 큰 정수 K를 골라, 부분집합의 모든 원소를 K로 나눈 나머지가 서로 같게 만드는 부분집합의 최대 크기를 구함.
- 제한 :
- 1. 집합 A의 크기 N : 1 <= N <= 20,000
- 2. 원소 값 A[i]:1 <= A[i] <= 10[9]
- 3. 모든 원소는 서로 다름

접근 전략

- 사용자로부터 입력받은 N개만큼 1 ~ 10^9 사이의 중복 없는 난수를 생성하고, 효율적인 계산을 위해 오름차순으로 정렬함.
- 정렬된 배열에서 인접한 두 수의 차이들을 구하고, 이들의 누적 최대공약수(GCD)를 계산하여 집합 전체가 하나의 K로 나누어떨어지는지 확인.

해결 방안

전체 집합 A의 모든 인접 원소 차이의 GCD가 1보다 크면 결과값은 N이 됨.
만약 GCD가 1이라면 공통된 약수가 없으므로, 조건을 만족하는 최대 부분집합은 단일 원소인 1이 됨.

알고리즘 설계 (수도코드)

```
시작
// 1. 집합 크기(N) 설정
사용자로부터 집합 A의 크기(n)를 입력받음

// 2. 랜덤 데이터 생성 (서로 다른 양의 정수)
만약 n이 0보다 크다면:
1부터 10억 사이의 숫자 중 중복 없이 n개를 무작위로 선택하여 집합(a) 생성
아니라면:
빈 집합 준비

// 3. 데이터 정합성 확인 및 정렬
만약 생성된 원소 개수가 부족하다면 오류 출력 후 종료
숫자들 사이의 거리를 계산하기 쉽게 집합(a)을 작은 순서대로 정렬(sort)

// 4. 핵심 로직: 모든 인접한 차이값의 최대공약수(GCD) 계산
만약 원소 개수가 2개 이상이라면:
초기_공약수 = (두 번째 숫자 - 첫 번째 숫자)

나머지 인접한 숫자들 사이의 거리(gap)를 하나씩 확인:
초기_공약수와 현재_거리(gap) 사이의 최대공약수를 구해 누적 갱신

// 5. 결과 판별 (나머지가 같게 만드는 K의 존재 여부)
최종_공약수가 1보다 크다면:
(모든 원소를 동일한 나머지로 만들 수 있으므로) 전체 크기 n 출력
아니라면:
(공통된 약수가 없으므로) 조건을 만족하는 최대 크기 1 출력

아니오 (원소가 1개뿐이라면):
최대 크기 1 출력  (공통 약수가 1뿐이라면 조건을 만족하는 부분집합은 원소가 1개 뿐이라서)
종료
```

문제 E

문제요약

- 목표: n개의 팀이 참여한 축구 리그에서 각 팀이 보고한 승점 리스트가 주어졌을 때, 이 점수들이 실제 리그 규칙 하에서 가능한지 유효성을 판별함.
- 규칙:
- 1. 모든 팀은 다른 모든 팀과 정확히 한 번씩 경기함 (총 경기 수: n(n - 1) / 2)
- 2.무승부는 없으며, 승리 시 1점, 패배 시 0점을 얻음
- 제한 : 팀의 개수 n : 2 <= n <= 10,000

접근 전략

가) 부분합 : k개의 팀의 전체 기준 승점의 합 ⇒ (k(k-1))/2 # 승점 합 ⇒ 최소승점
나) 전체합 : 전체 n개 팀 승점 합 = (n(n-1))/2 # 무승부X, 승리 +1점, 패배 0점 조건

해결 방안

입력받은 승점 리스트를 오름차순으로 정렬한 후 반복문을 사용하여 승점을 누적합계에 저장하는 동시에 if문으로 부분합 조건을 확인한다.

알고리즘 설계 (수도코드)

```
시작
//1. for문을 사용해 0부터 n - 1까지 정수 i를 1씩 증가시킨다.

//2. sum에 += 대입연산자를 사용하여 score[i]를 더한다.
      k에 i + 1을 대입한다.
//3. if문을 사용하여 sum이 (k*(k-1)/2)보다 작으면(가 조건에 반대되는 조건) isValid에
     False를 대입하고 break를 사용하여 for문에서 빠져나간다.

//4. if문을 사용하여 isValid와 (누적합 = (n*(n-1))//2)가 둘 다 참인지 확인한다 # isValid는
가)조건, 방정식은 나)조건이 참인지 확인하는 변수이다.
(만약 두 변수가 참이면 1을 반환하고 거짓이라면 -1을 반환한다.)
```

문제 F

문제요약

- 목표 : 구멍을 제외한 실제 츠지 부피를 기준으로 모든 슬라이스를 동일한 부피로 나누기
- 조건
- 1. 크기 : 100 X 100 X 100 치즈
- 2. 내부에 겹치지 않는 구형 구멍 n개
- 3. z축 방향으로 s개의 슬라이스로 절단

접근 전략

- 길이가 아니라 부피 기준 분할 문제
- 구멍 때문에 밀도가 일정하지 않음

핵심 개념 (누적부피 함수)

- 누적부피(z) = z까지 존재하는 실제 치즈 부피
- 전체 직육면체 부피 - 구멍(구) 부피
- z 증가 → 부피 증가 (단조 증가)

핵심 개념 (구멍(구) 처리 방법)

- 전혀 안겹침 : 부피 영향 없음
- 완전히 포함됨 : 전체 구 부피 사용
- 일부만 포함됨 : 구가 잘린 상태 → 구면 캡 사용

알고리즘 설계 (수도코드)

```
입력:
    n, s
    n개의 구 (r, x, y, z)

함수 구부피(r):
    반환 (4/3)πr³

함수 캡부피(h, r):
    반환 πh²(r - h/3)

함수 누적부피(z):
    전체부피 = 100 × 100 × z
    구멍부피 = 0

    모든 구에 대해 반복:
    아래 = cz - r
    위 = cz + r

    # 1. 안 겹침
if 위 ≤ 0 또는 아래 ≥ z:
continue

# 2. 완전 포함
if 아래 ≥ 0 그리고 위 ≤ z:
구멍부피 += 구부피(r)

# 3. 일부 포함
else:
    시작 = max(0, 아래)
    끝 = min(z, 위)

    h1 = 시작 - 아래
    h2 = 끝 - 아래

    구멍부피 += 캡부피(h2, r) - 캡부피(h1, r)

반환 전체부피 - 구멍부피

----------------------------------------
함수 다음높이 (이전높이, 목표부피):

    left = 이전높이
    right = 100

    반복 (정밀도 확보):
    mid = (left + right) / 2

부피 = 누적부피(mid) - 누적부피(이전높이)

    if 부피 < 목표부피:
        left = mid
    else:
        right = mid

    반환 mid
----------------------------------------

예인:

전체부피 = 누적부피(100)
목표부피 = 전체부피 / s

이전 = 0

for i = 1 ~ s:
현재 = 다음높이(이전, 목표부피)

출력 (현재 - 이전)
이전 = 현재
```

문제 G

문제요약

- 목표: n개의 섬이 m개의 다리로 연결되어 있을 때, 어떤 다리 하나가 끊어지더라도 모든 섬이 계속 연결된 상태를 유지하기 위해 추가해야 하는 최소 다리 수를 구함.
- 제한 :
- 1. 섬의 수 n:3 <= n <= 100,000
- 2. 다리의 수 m:n - 1 <= m <= 2n - 3
- 3. 섬들은 볼록 다각형의 꼭짓점 위치에 있으며, 다리는 서로 교차하지 않음

접근 전략

- '(leaf_count + 1) / 2'를 중점으로 고립된 섬에서 막다른 그룹 판단 보강이 반드시 필요하거나, 다른 말단과 연결대상이 될 경우 count를 증가시키며, 이에 반대 상황일 경우 증가시키지 않는다.
- 해당 공식은 종이 한 장을 한 번 자르면 종이가 두 장이 되는 방식에서 착안하였다.

해결 방안

끝과 끝을 잇기 : 지도의 말단들을 서로 연결해버리면, 전체 지도가 커다란 원형 형태가 되는 것을 중점으로 문제풀이.

알고리즘 설계 (수도코드)

```
// 1. 지도 정보 입력 및 초기화
공원의 섬 개수(N), 현재 다리 개수(M)를 입력
대량의 데이터를 빠르게 처리하기 위해 입력 설정 최적화

// 2. 섬들 사이의 연결 지도 그리기
섬_연결_리스트(graph) 생성 (빈 목록)
M번 반복하며 어떤 섬들이 서로 이어져 있는지 기록

// 3. 문제가 되는 외길(=절단교) 찾아내기
다리 하나를 끊었을 때 지도가 두 그룹으로 나뉘는 모든 문제가 되는 외길을 찾아냄
해당 다리가 끊기면 섬의 절반이 고립되는 위험 지점들을 식별함

// 4. 안전 구역 팀 묶기 및 그룹화
절단교를 제외하고 자유롭게 오갈 수 있는 섬들을 한 그룹으로 묶음
안전_그룹_목록(groups) 생성

// 5. 핵심 로직
막다른 그룹을(Leaf) 찾기
막다른_그룹_개수(leaf_count) = 0 설정
각 그룹을 하나씩 확인:
만약 해당 팀과 연결된 외길이 단 1개뿐이라면:
이 팀을 고립 위험이 큰 '막다른 그룹'으로 판단하여 leaf_count를 1 증가시킴
그렇지 않을 경우 leaf_count 증가시키지 않음:

// 6. 결과 출력
막다른 곳 2개를 이을 때마다 2곳의 고립 위험이 동시 해결가능
최종 계산식 '(leaf_count + 1) / 2'을 통해 필요한 최소로 추가할 다리 수 출력
종료
```